Song Recommendation Baseline Report

This report compares two basic ways to recommend songs: MostPopular and ItemKNN. The aim is to suggest songs that users have not listened to before.

1. Summary

ItemKNN did better than MostPopular in our tests. Its NDCG@10 score was 0.0299, compared with 0.0151 for MostPopular. It also gave useful top-10 recommendations to 17.3% of users, compared with 8.8% for MostPopular. The test included 7,202 users. ItemKNN recommended 7,944 different songs across users, while MostPopular recommended only 154.

Our ItemKNN result was still lower than the score in the Yambda research paper. We explain some possible reasons below.

2. Data and setup

We used the Yandex Yambda-50M listening data from Hugging Face. It has about 46.5 million listens over around 300 days. We counted a listen when more than 50% of a song was played. After cleaning, there were about 9.02 million user-song pairs, from 9,207 users and about 630,000 songs.

We split the data by time. Earlier listens were used for training, then we used 7 days for checking settings and another 7 days for final testing. There was a 30-minute gap before each period. We only tested songs the user had not listened to before.

We kept songs that had at least 20 different listeners in the training data. This left 67,931 songs. We used Python 3.10, DuckDB, NumPy and scikit-learn. The whole process can be run with python scripts/run_baselines.py and takes about 10 minutes on an 8 GB laptop.

3. Models we tested

MostPopular recommends songs that were listened to by the most people in the training data. It leaves out songs the user already knows, but most users get nearly the same suggestions.

ItemKNN looks for songs that have similar listeners. It then recommends songs similar to the ones a user has played. We tried k values of 20, 50, 100 and 200. A value of 20 worked best in the validation test.

4. Test results

Model	NDCG@10	Recall@10	Precision@10	Hit@10	Coverage@10
MostPopular	0.0151	0.0067	0.0133	0.088	0.002
ItemKNN (k=20)	0.0299	0.0116	0.0272	0.173	0.117

For the top 50 songs, MostPopular scored 0.0158 on NDCG, 0.0179 on recall, 0.203 on hit rate and 0.005 on coverage. ItemKNN scored 0.0354, 0.0427, 0.422 and 0.328 on the same measures.

NDCG@10 is our main measure. In simple terms, a higher score means the useful songs are placed higher in the list. Coverage tells us how much of the song catalog gets recommended. Only 72.9% of test listens involved songs in our chosen catalog, which limits the scores.

5. Why our scores are different from the paper

Model	Our NDCG@10	Paper NDCG@10
MostPopular	0.0151	0.0186
ItemKNN	0.0299	0.0781

Our MostPopular score is fairly close to the paper, but our ItemKNN score is only about 38% of the paper's result. We think this could be because we only test first-time listens, while the paper may also count repeated listens. We also use a 7-day test period instead of 1 day, remove less popular songs, and test fewer k settings. These are possible reasons, not proven explanations.

We also noticed that MostPopular dropped from 0.0191 in validation to 0.0151 in testing, while ItemKNN stayed more stable. Changing k did not make much difference. ItemKNN worked better for lighter listeners (0.042 NDCG@10) than heavy listeners (0.020). It also still recommended a lot of popular songs: 80% of its top-10 picks came from the most popular 10% of songs.

6. What we will do next (October 20–30)

Start: Build two more models, iALS and BPR. Try a popularity model that gives more weight to recent listens. Add 95% confidence intervals to check whether score differences matter. Run another test that allows repeat listens. Add the missing tests folder and mark the code release.

Stop: Do not spend more time testing many extra k values because the results hardly changed. Avoid running the full dataset for every small experiment; use the 20% user sample while developing.

Continue: Keep the same time-based data split, first-time-song testing, fixed data version and library versions. Keep checking how user activity and song popularity affect results.

Overall, ItemKNN is a better starting point than MostPopular for this project, but there is still room to improve it.